

Análisis de Algoritmos 2022/2023

Práctica 3

Íñigo Álvarez y Enrique Gómez. Grupo 1271

1. Introducción.

En esta práctica se trata el desarrollo y el análisis de algoritmos de búsqueda sobre diccionarios. Se determinará experimentalmente el tiempo medio de búsqueda de los elementos en un diccionario que emplea como tipo de datos una tabla.

2. Objetivos

2.1 Apartado 1

Para el apartado 1, hemos realizado un TAD de un diccionario, en el cual hemos tenido que crear una función para inicializarlo y reservar memoria, otra para liberarla, otra para introducir un dato en la tabla dependiendo de si la queremos ordenada o no, otra para introducir una lista de datos en la tabla y por último una para buscar el índice en la que está un número determinado. Esto último puede realizarse mediante 3 formas: con una búsqueda lineal, una búsqueda binaria y una auto lineal.

2.2 Apartado 2

En el apartado 2, hemos creado funciones necesarias para medir los tiempos de ejecución para las 3 distintas búsquedas que hemos realizado en el apartado 1. Además se contarán el número de operaciones básicas máxima, mínima y media para un mismo número de elementos en la tabla y todo esto se imprimirá en un archivo seleccionado, el cual se creará en caso de que este no esté creado. Como dice el enunciado, estos tiempos serán para tablas de entre 1000 y 10000 elementos.

3. Herramientas y metodología

Aquí ponéis qué entorno de desarrollo (Windows, Linux, MacOS) y herramientas habéis utilizado (Netbeans, Eclipse, gcc, Valgrind, Gnuplot, Sort, uniq, etc) y qué metodologías de desarrollo y soluciones al problema planteado habéis empleado en cada apartado. Así como las pruebas que habéis realizado a los programas desarrollados.

3.1 Apartado 1

Hemos utilizado el entorno de desarrollo Linux, con Visual Studio Code como herramienta para programar. Para compilar el programa hemos utilizado gcc así como algunas flags como -Wall, -ansi o -pedantic. Valgrind lo hemos empleado para comprobar si la memoria de nuestro programa estaba correctamente usada. Para comprobar el funcionamiento de este apartado hemos usado el test que hay en el Makefile `exercise1_test`, cambiando sus datos para comprobar posibles fallos y cambiando el nombre de la función a comprobar asegurar el correcto funcionamiento de las 3 funciones de búsqueda.

3.2 Apartado 2

En el apartado 5 también se ha utilizado el entorno de desarrollo Linux y las herramientas Visual Studio Code, gcc o valgrind. También hemos empleado la herramienta Gnuplot para generar gráficas con los datos obtenidos. La prueba principal ha sido el test del makefile (`exercise2_test`) cambiando los datos para generar distintas pruebas, pero siempre con datos de tablas de 1000 a 10000 elementos.

4. Código fuente

4.1 Apartado 1

```
PDICT init_dictionary (int size, char order)
{
    DICT *dict;
    int i = 0;

    if (size < 0 || order < 0)
    | return NULL;
    dict = (PDICT)malloc(sizeof(DICT) * 1);
    if (!dict)
    | return NULL;
    dict->n_data = 0;
    dict->order = order;
    dict->size = size;
    dict->table = (int *)malloc(sizeof(int) * size);
    if (!dict->table)
    {
        free(dict);
        return NULL;
    }
    while (i < size)
    {
        dict->table[i] = 0;
        i++;
    }
    return dict;
}
```

```
void free_dictionary(PDICT pdict)
{
    if (!pdict)
    | return;
    free(pdiction->table);
    free(pdiction);
    return;
}
```

```
int insert_dictionary(PDICT dict, int key)
{
    int j=0, A=0, count=0;
    if (key < 0 || !dict->table || dict->n_data < 0 || dict->order < 0 || !dict->size)
    {
        | return ERR;
    }

    if (dict->n_data == dict->size - 1)
    {
        dict->table = realloc(dict->table, ((dict->size) * 4) + 400);
        if (!dict->table)
        {
            | return ERR;
        }
        dict->size = dict->size + 100;
    }

    dict->table[dict->n_data] = key;
    dict->n_data++;

    if (dict->order == NOT_SORTED)
    {
        | return count;
    }
    A = dict->table[dict->n_data - 1];
    j = (dict->n_data - 2);
    while (j >= 0 && dict->table[j] > A)
    {
        dict->table[j + 1] = dict->table[j];
        j--;
        count++;
    }
    dict->table[j + 1] = A;
    return count;
}
```

```
int massive_insertion_dictionary (PDICT pdict,int *keys, int n_keys)
{
    int i = 0;
    int c = 0;
    int caux = 0;

    if (!pdict || !keys || n_keys < 1)
    | return ERR;
    while (i < n_keys)
    {
        caux = insert_dictionary(pdict, keys[i]);
        if (caux == ERR)
        | return ERR;
        c += caux;
        i++;
    }
    return c;
}
```

```
int search_dictionary(PDICT pdict, int key, int *ppos, pfunc_search method)
{
    if (!pdict || key < 0 || *ppos < 0 || !method)
    | return ERR;
    return method(pdiction->table, 0, pdiction->n_data - 1, key, ppos);
}
```

```

int bin_search(int *table,int F,int L,int key, int *ppos)
{
    int ind_mid=0, mid=0, count=0;

    if (F > L || F < 0 || !table || key < 0 || !ppos)
    {
        return ERR;
    }
    while (F <= L)
    {
        ind_mid = (F + L) / 2;

        mid = table[ind_mid];
        count++;
        if (key == mid)
        {
            (*ppos) = ind_mid;
            return count;
        }

        if (key < mid)
        {
            L = ind_mid - 1;
        }
        else
        {
            F = ind_mid + 1;
        }
    }
    (*ppos) = NOT_FOUND;
    return count;
}

```

```

int lin_search(int *table,int F,int L,int key, int *ppos)
{
    int i = F;
    int c = 0;

    if (!table || L < F || key < 0 || !ppos || F < 0)
    {
        return ERR;
    }
    while (i <= L)
    {
        c++;
        if (table[i] == key)
        {
            (*ppos) = i;
            return c;
        }
        i++;
    }
    (*ppos) = NOT_FOUND;
    return c;
}

```

```

int lin_auto_search(int *table,int F,int L,int key, int *ppos)
{
    int i = F;
    int c = 0;
    int aux = 0;

    if (!table || L < F || key < 0 || !ppos || F < 0)
    {
        return ERR;
    }
    while (i <= L)
    {
        c++;
        if (i == 0 && table[i] == key)
        {
            (*ppos) = i;
            return c;
        }
        if (table[i] == key)
        {
            (*ppos) = i;
            aux = table[i];
            table[i] = table[i - 1];
            table[i - 1] = aux;
            return c;
        }
        i++;
    }
    (*ppos) = NOT_FOUND;
    return c;
}

```

4.2 Apartado 2

```
short average_search_time(pfunc_search method, pfunc_key_generator generator, char order, int N, int n_times, PTIME_AA ptime)
{
    POICT dictionary;
    int *permutation;
    int i = 0;
    clock_t start, end;
    int count = 0;
    int count_min = 0;
    double count_total = 0;
    int count_max = 0;
    double a = 0;
    int ppos;
    int *keys_table = NULL;

    if (!method || !generator || N < 0 || ptime == NULL || n_times < 0 || order < 0 || order > 1)
        return ERR;

    dictionary = init_dictionary(N, order);
    if (!dictionary)
        return ERR;

    permutation = generate_perm(N);
    if (permutation == NULL){
        free_dictionary(dictionary);
        return ERR;
    }

    count = massive_insertion_dictionary(dictionary, permutation, N);
    if (count == ERR){
        free_dictionary(dictionary);
        free(permutation);
        return ERR;
    }

    keys_table = malloc(sizeof(int) * (n_times * N));
    if (keys_table == NULL){
        free_dictionary(dictionary);
        free(permutation);
        return ERR;
    }
}
```

```
generator(keys_table, n_times * N, N);
if(keys_table){
    free(keys_table);
    free_dictionary(dictionary);
    free(permutation);
    return ERR;
}

for (i = 0; i < n_times * N; i++)
{
    start = clock();
    count = search_dictionary(dictionary, keys_table[i], 4ppos, method);
    if (count == ERR || ppos < 0 || ppos > N)
    {
        free(keys_table);
        free_dictionary(dictionary);
        free(permutation);
        return ERR;
    }
    end = clock();
    if (!end || !start)
    {
        free(keys_table);
        free_dictionary(dictionary);
        free(permutation);
        return ERR;
    }
    count_total += count;
    if (count <= count_min || count_min == 0)
        count_min = count;
    if (count >= count_max)
        count_max = count;
    a += ((end - start));
}

ptime->time = a / (n_times * N);
ptime->n_elems = n_times * N;
ptime->N = N;
ptime->average_ob = count_total / (n_times * N);
ptime->min_ob = count_min;
ptime->max_ob = count_max;

free_dictionary(dictionary);
free(keys_table);
free(permutation);
return OK;
}
```

```
short generate_search_times(pfunc_search method, pfunc_key_generator generator, int order, char* file, int num_min, int num_max, int incr, int n_times)
{
    PTIME_AA time;
    int i, N, j;

    if (!method || !file || !generator || num_min < 0 || num_max < 0 || num_min > num_max || incr <= 0 || n_times < 0 || order < 0 || order > 1)
        return ERR;
    N = ((num_max - num_min) / incr) + 1;
    time = malloc(sizeof(TIME_AA) * N);
    if (!time)
        return ERR;
    for (i = 0, j = num_min; i < N; i++)
    {
        if (average_search_time(method, generator, order, j, n_times, &time[i]) == ERR)
        {
            free(time);
            return ERR;
        }
        j += incr;
        if (j > num_max + 1)
            break;
    }
    if (save_time_table(file, time, i) == ERR)
    {
        free(time);
        return ERR;
    }
    free(time);
    return OK;
}
```

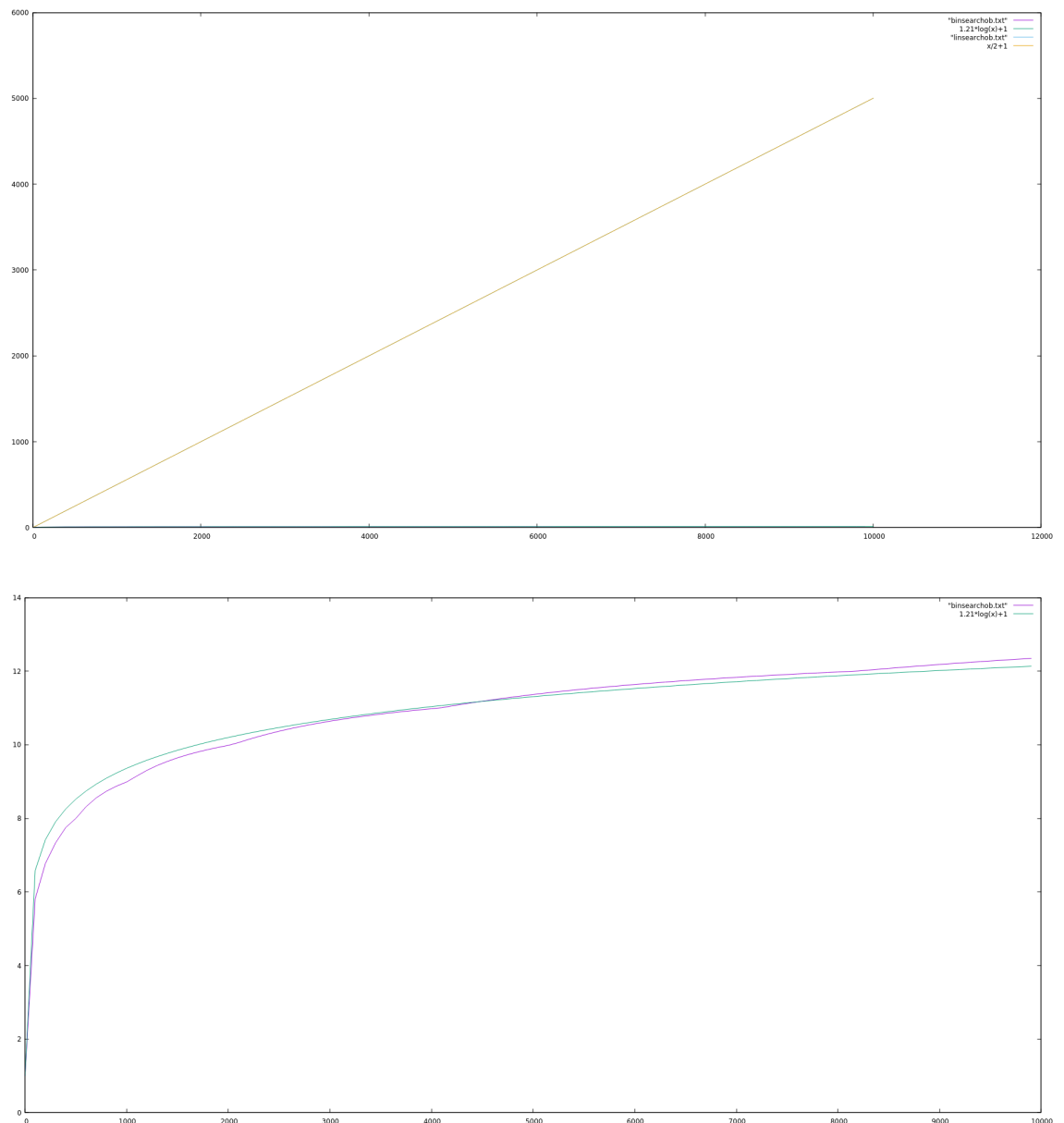
5. Resultados, Gráficas

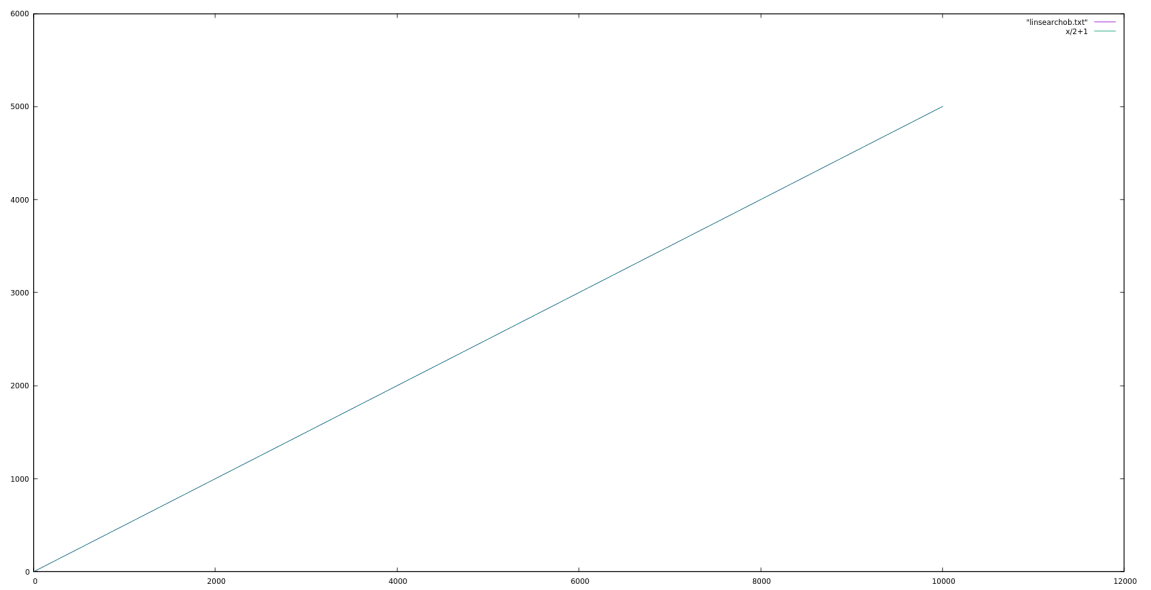
5.1 Apartado 1

En el apartado 1, la solución obtenida es la posición obtenida para una de las búsquedas, elegida en el código, en la cual imprime por pantalla la clave que se está buscando en la tabla, el índice en el que se encuentra la posición y el número de operaciones básicas realizadas para encontrar la clave. Nosotros hemos comprobado que la solución era correcta imprimiendo por pantalla la tabla para ver si estaba en esa posición.

5.2 Apartado 2

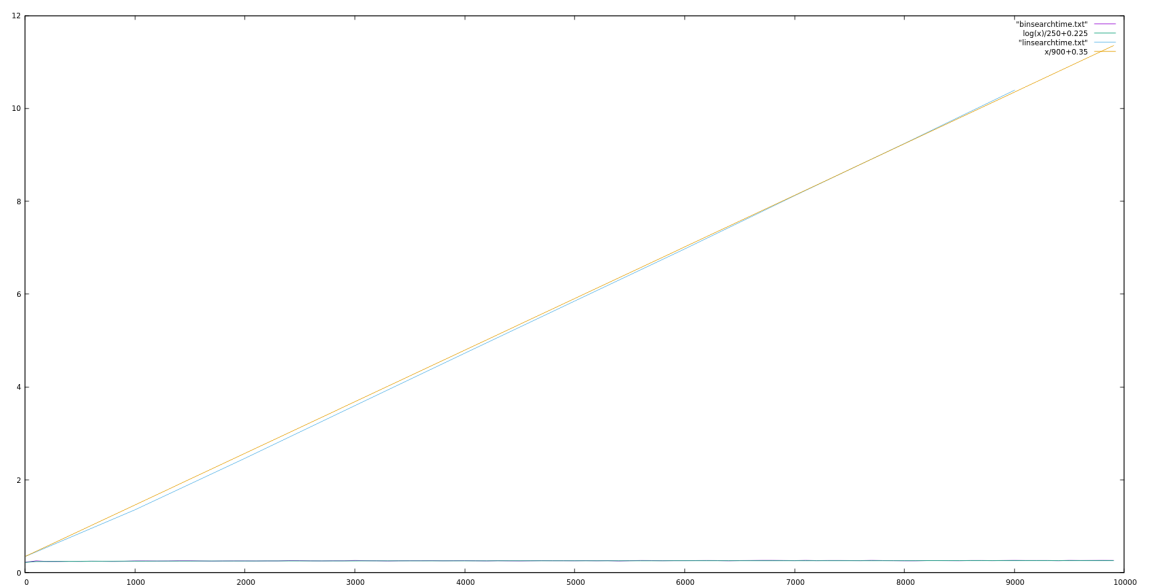
Gráfica comparando el número promedio de OBs entre la búsqueda lineal y la búsqueda binaria, comentarios a la gráfica:

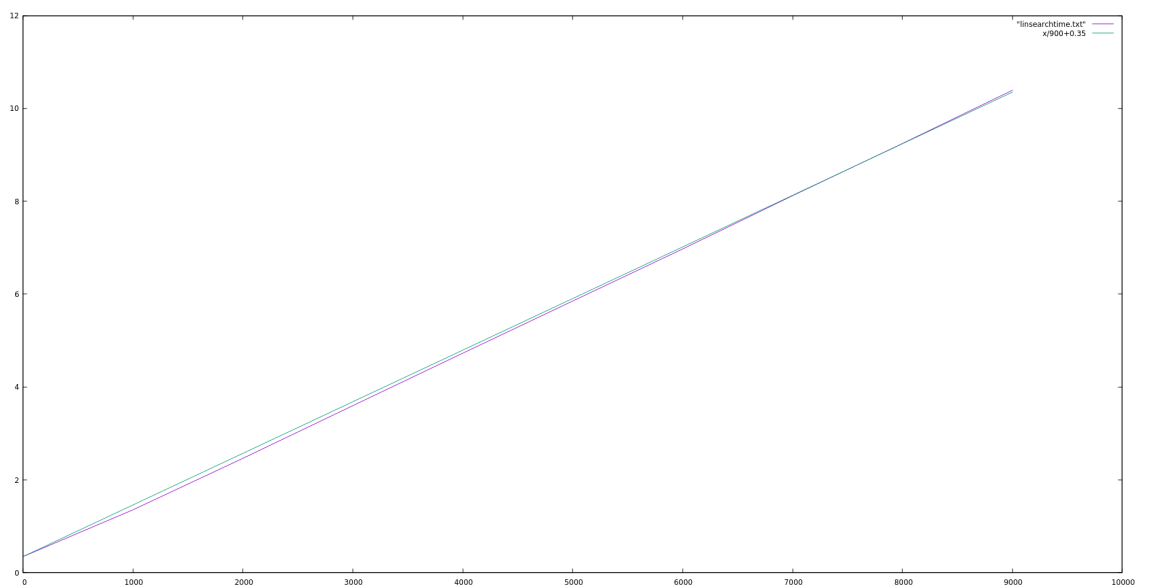
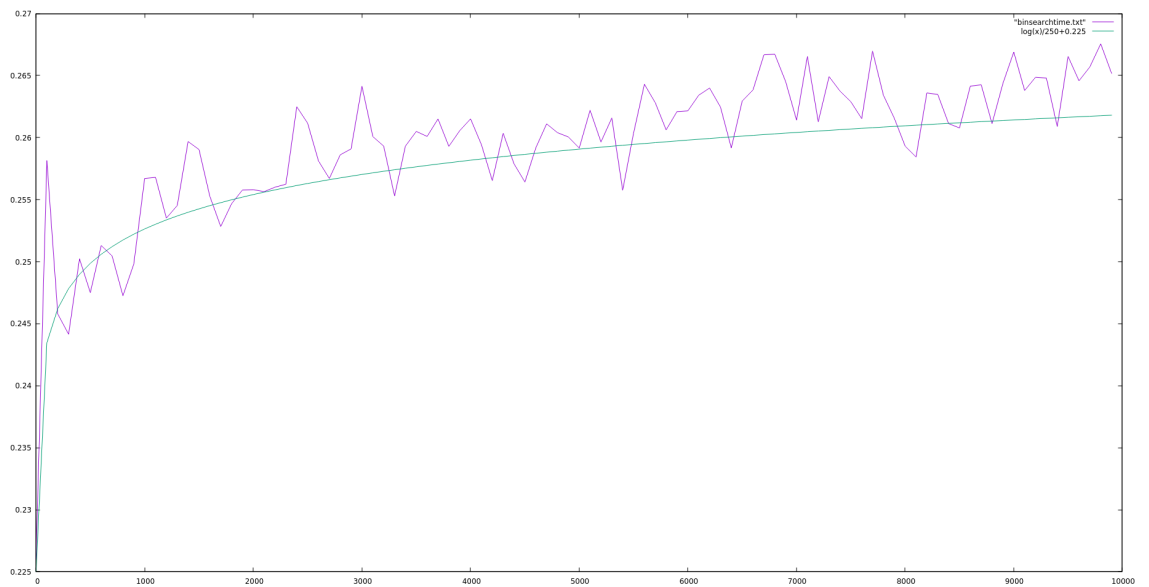




Como podemos apreciar en la gráfica es mucho más óptima la búsqueda binaria. Su complejidad es $\log(n)$ mientras que la de la lineal es $n/2$.

Gráfica comparando el tiempo promedio de reloj entre la búsqueda lineal y la búsqueda binaria, comentarios a la gráfica:

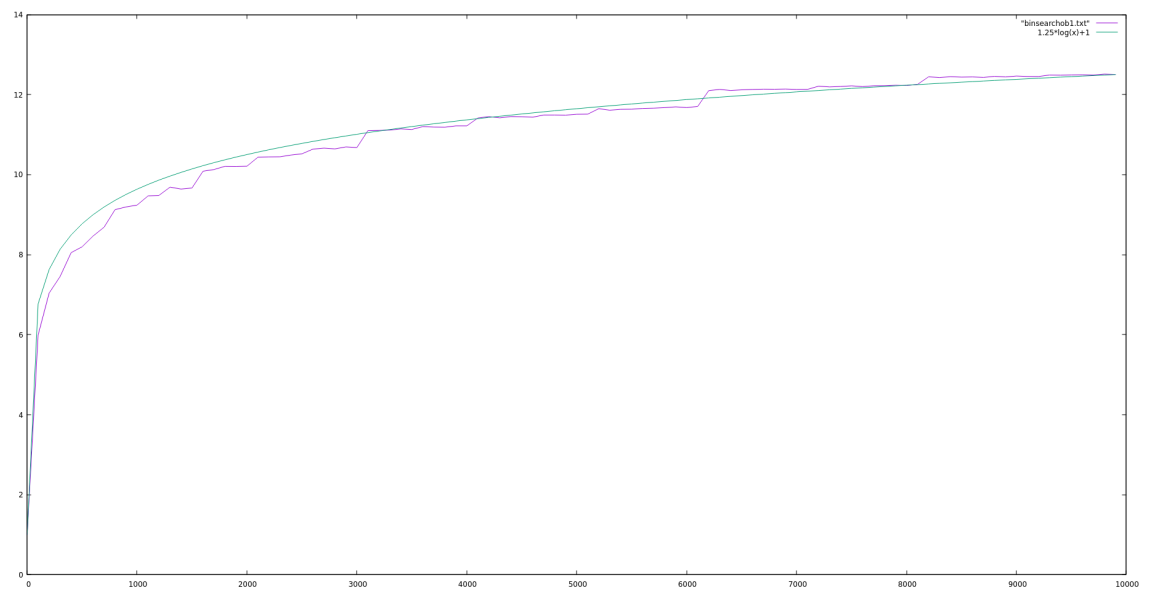
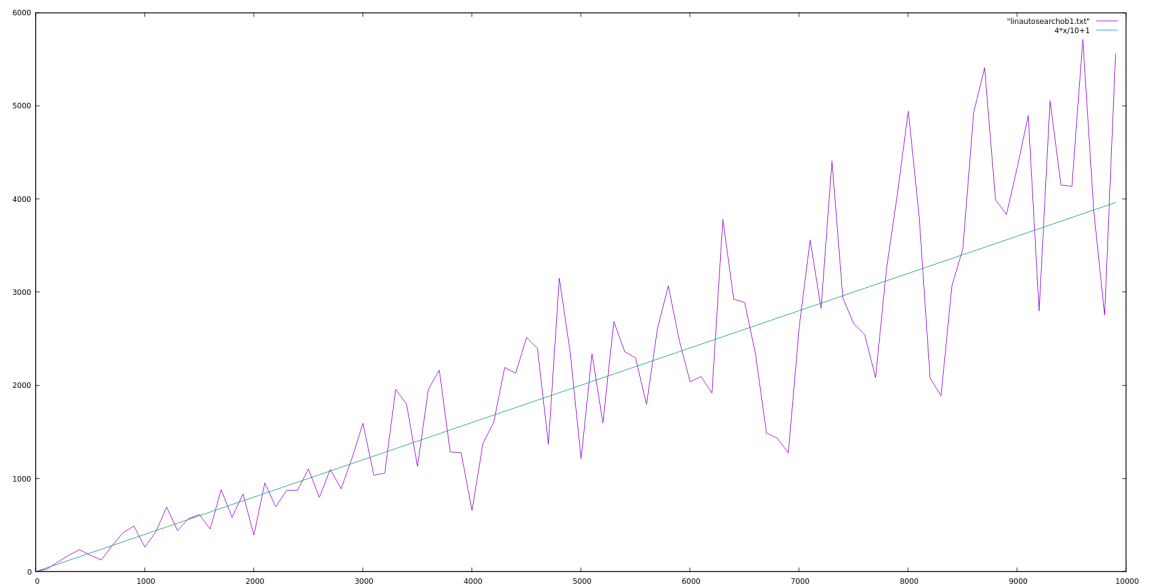
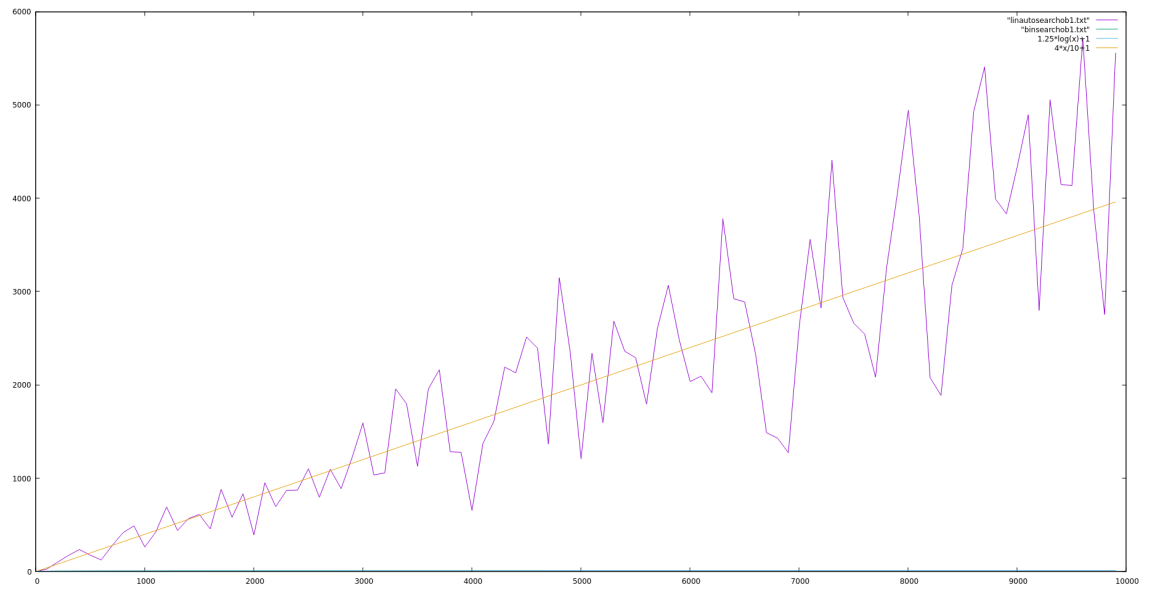




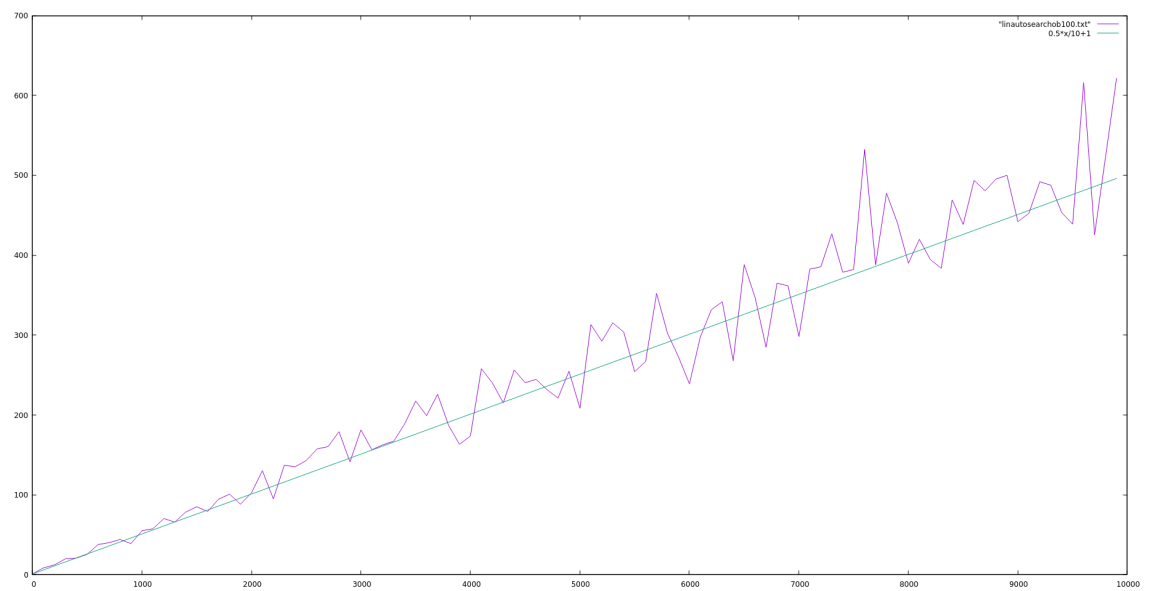
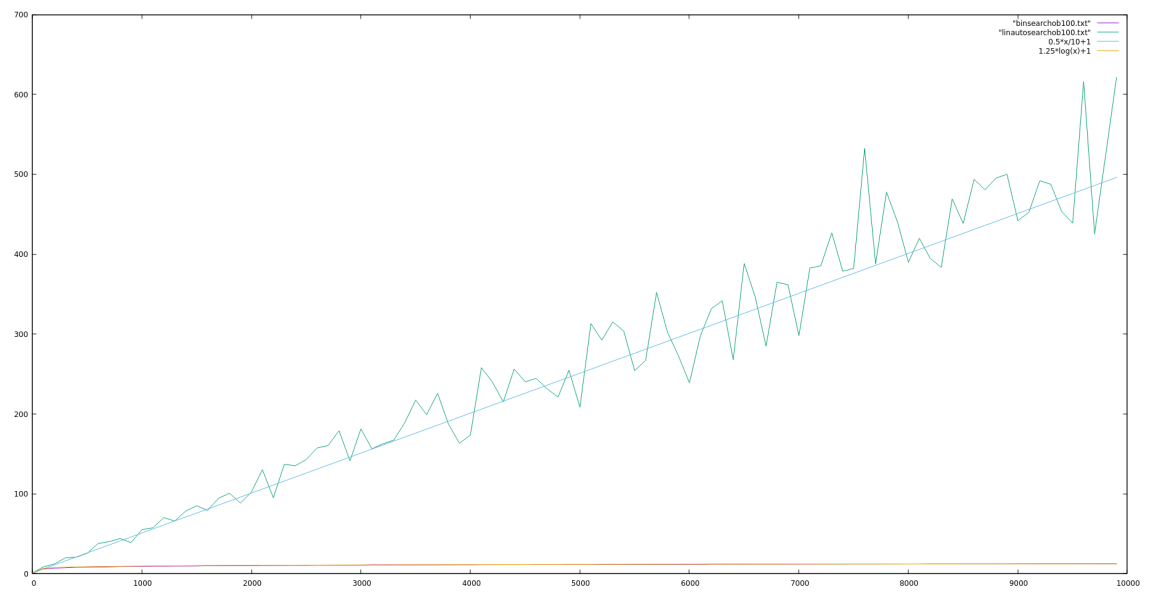
En cuanto al tiempo, al igual que el número de operaciones básicas, es mucho más óptima la búsqueda binaria (complejidad $\log(n)$) que la búsqueda lineal.

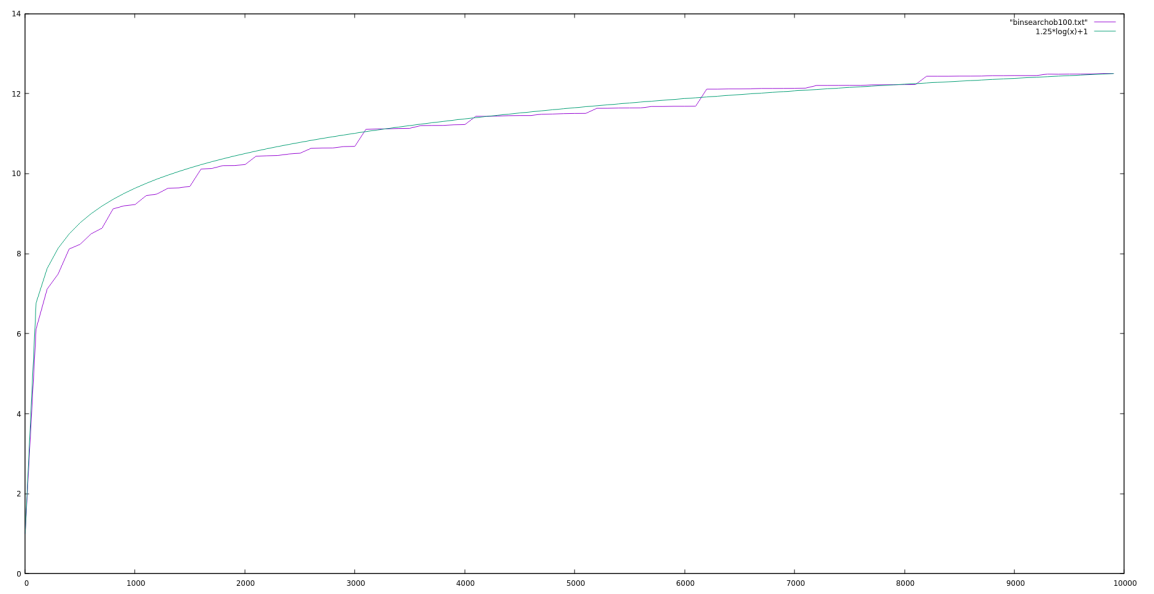
Gráfica comparando el número promedio de OBs entre la búsqueda binaria y la búsqueda lineal auto organizada (para los valores de $n_times=1, 100$ y 10000), comentarios a la gráfica:

N_times 1:

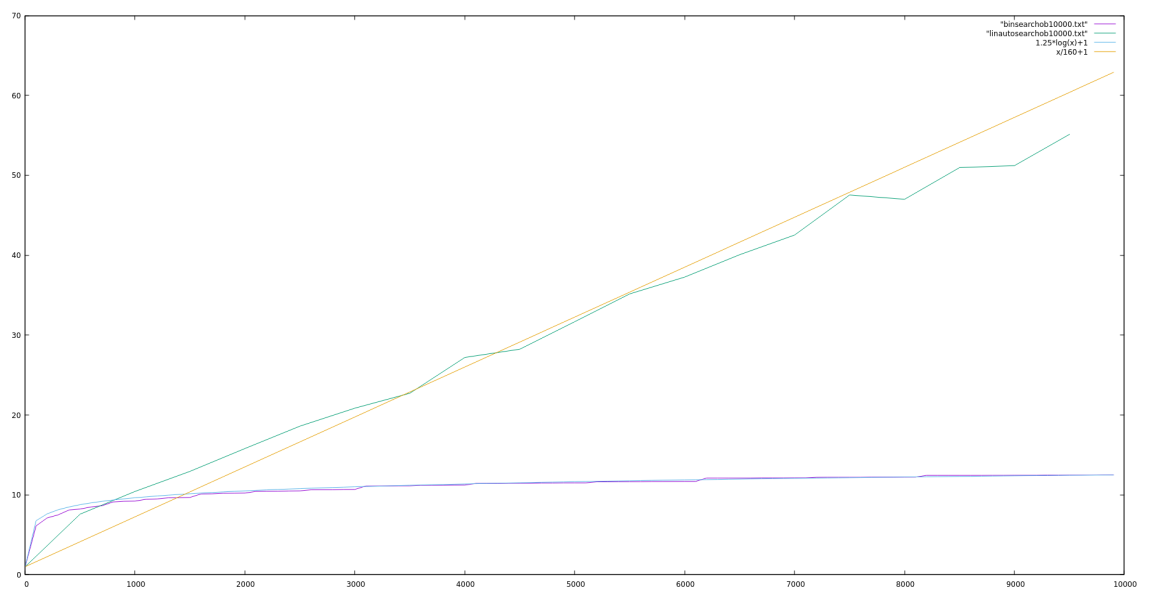


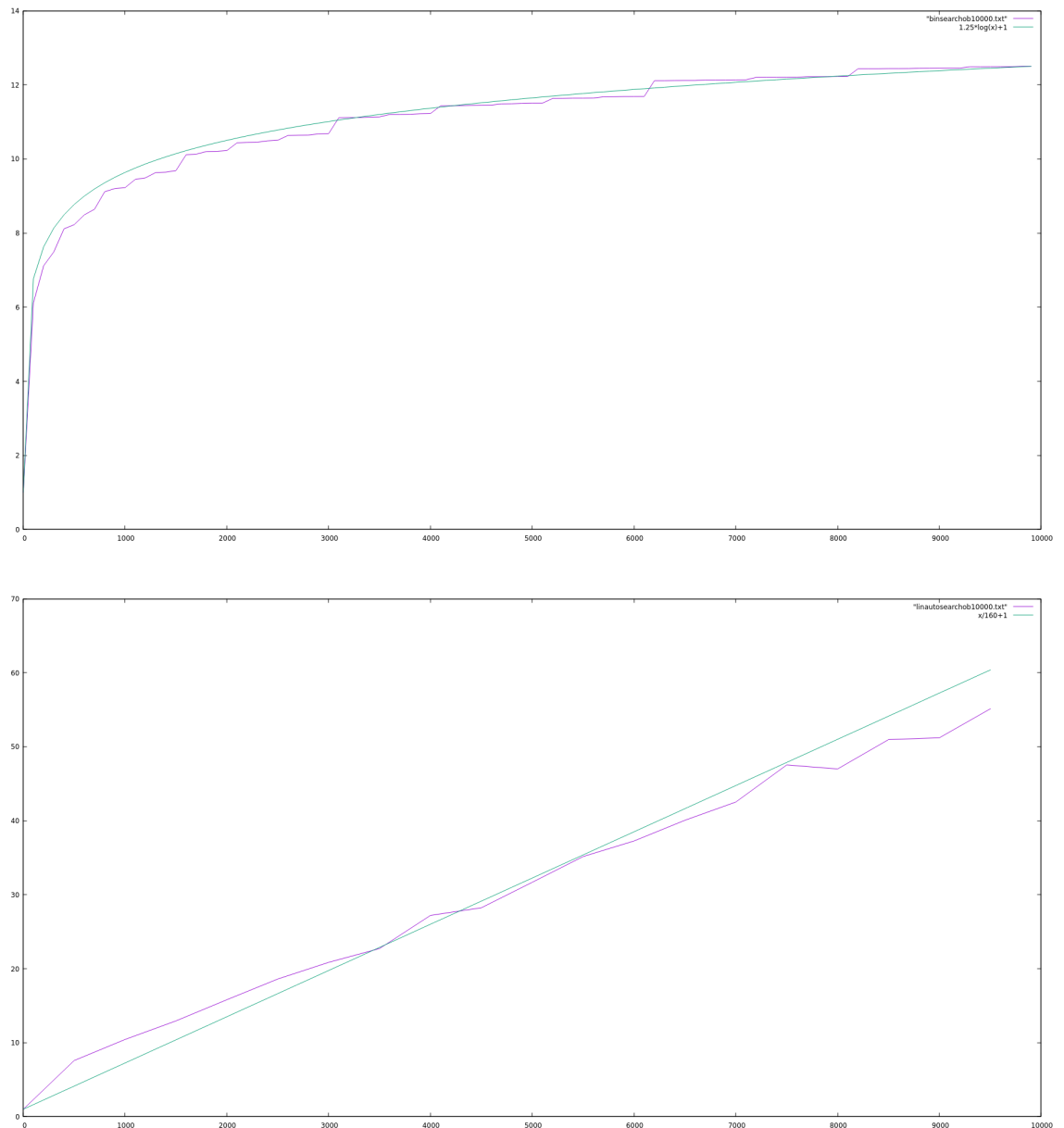
N_times 100:





N_times 10000:

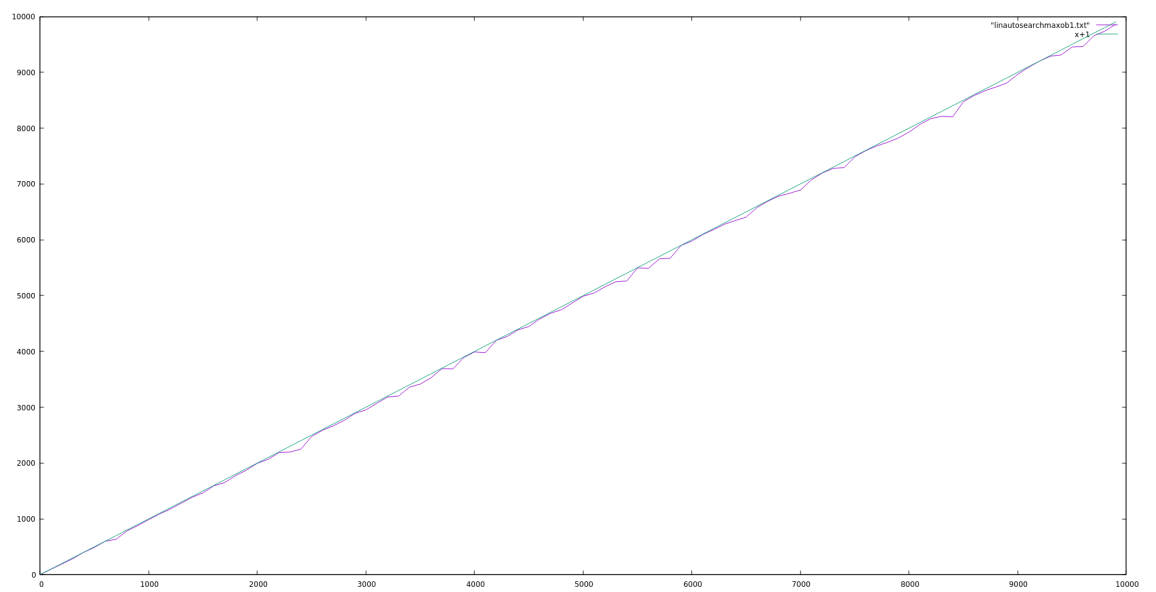
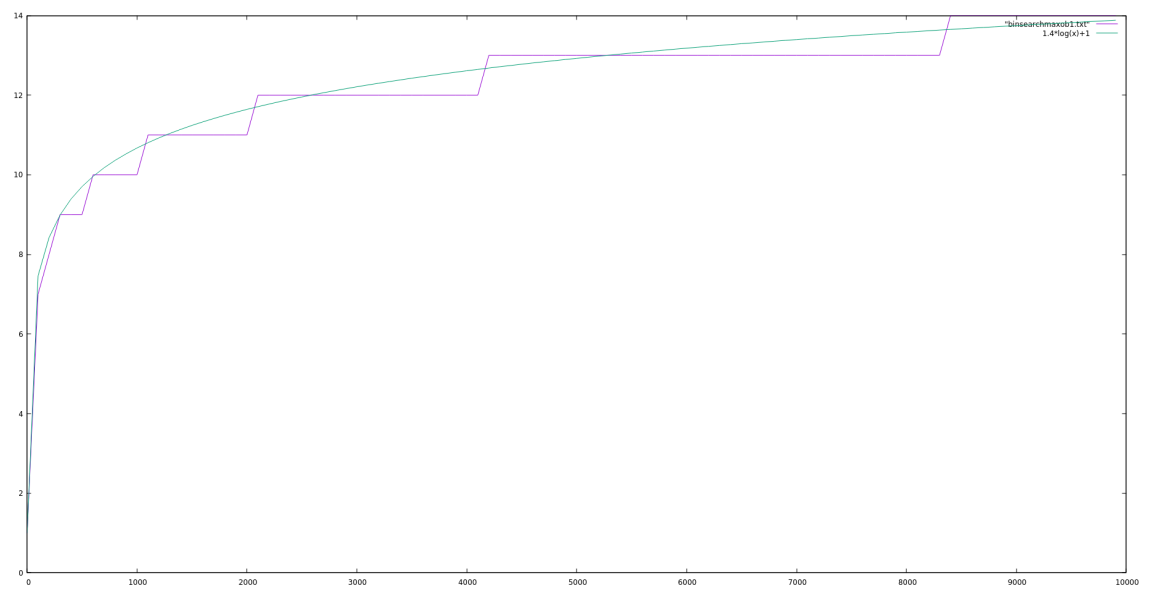
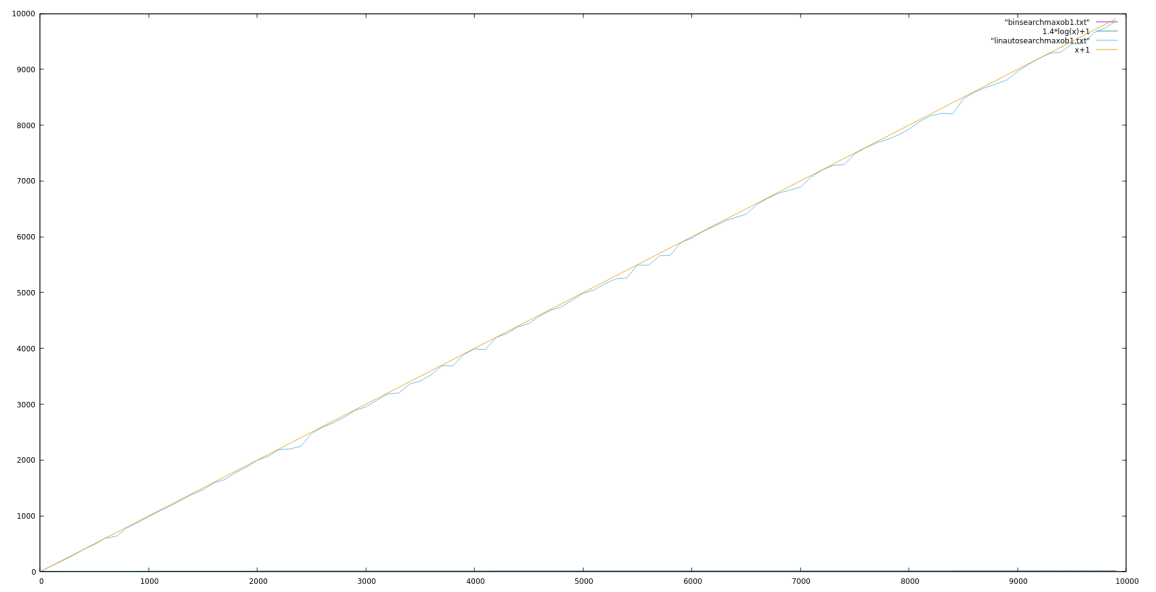




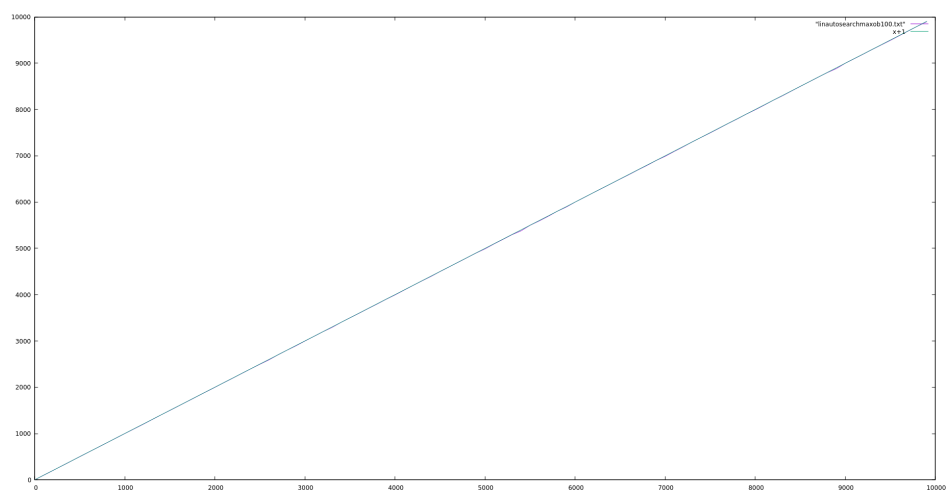
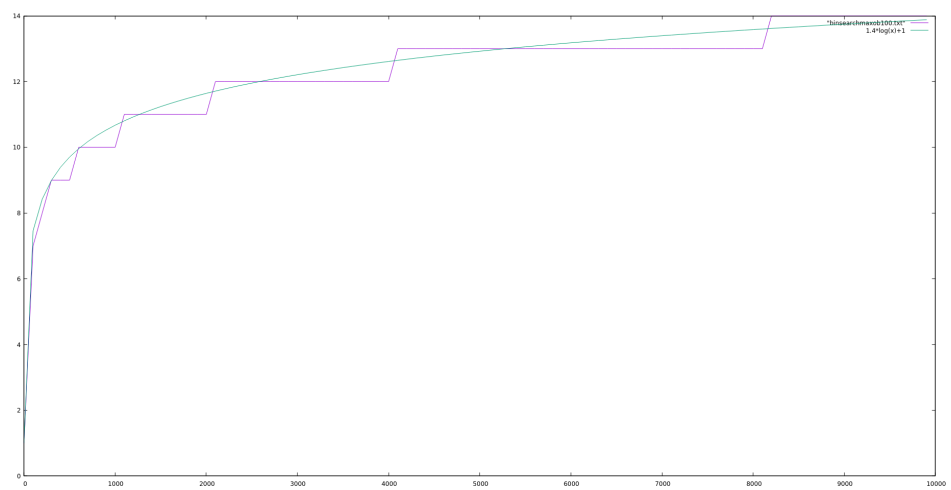
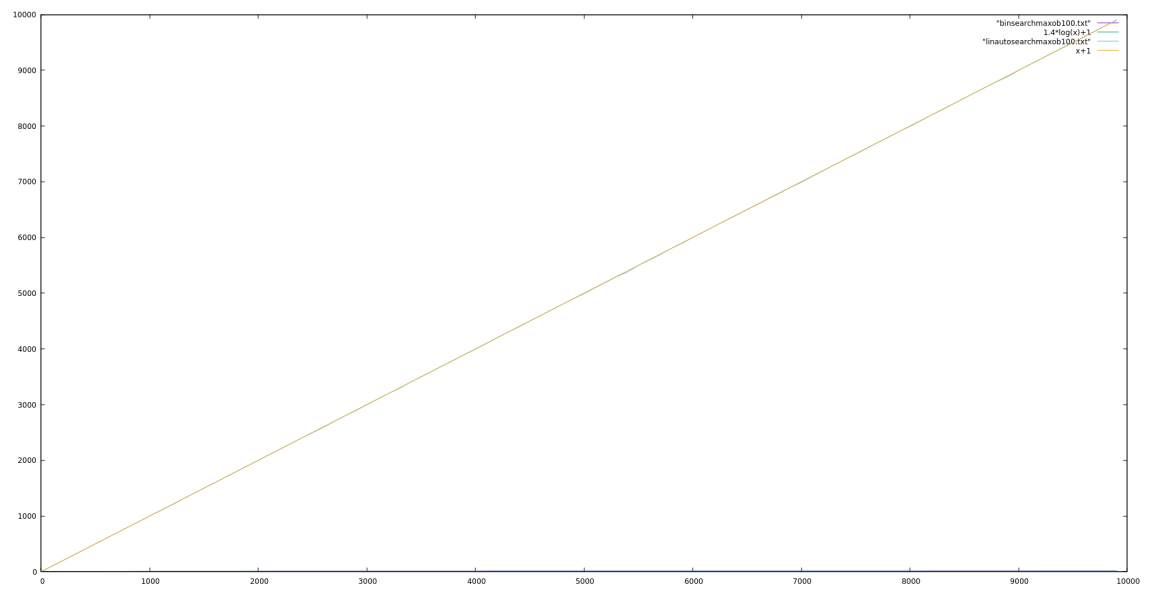
Como podemos apreciar para el número promedio de OBs cuanto mayor sea el n_times mejor es el rendimiento de la búsqueda auto lineal. Esto se debe a que con la elección potencial de claves se van a ir eligiendo mayoritariamente los mismos números y al ir quedando estos números al principio de la tabla cada vez los encontraremos más rápido. La búsqueda binaria sin embargo no se ve afectada por el n_times porque la tabla siempre va a estar ordenada. En todos los casos que hemos probado es más eficiente la búsqueda binaria, sin embargo si pusieramos un n_times grandísimo la búsqueda auto lineal sería más eficiente.

Gráfica comparando el número máximo de OBs entre la búsqueda binaria y la búsqueda lineal auto organizada (para los valores de $n_times=1$, 100 y 10000), comentarios a la gráfica:

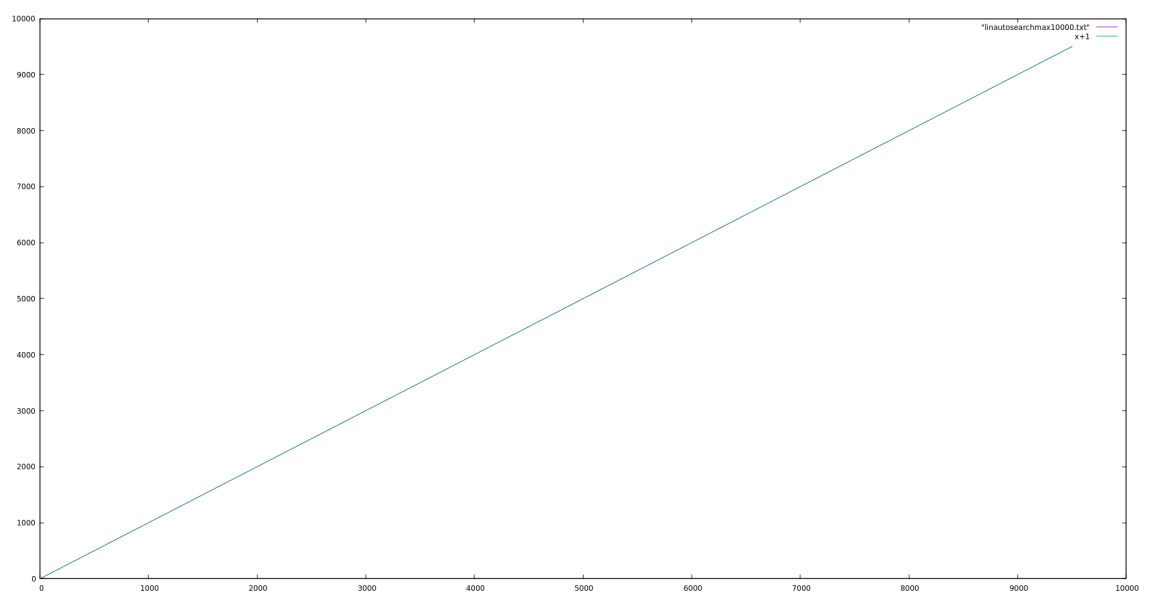
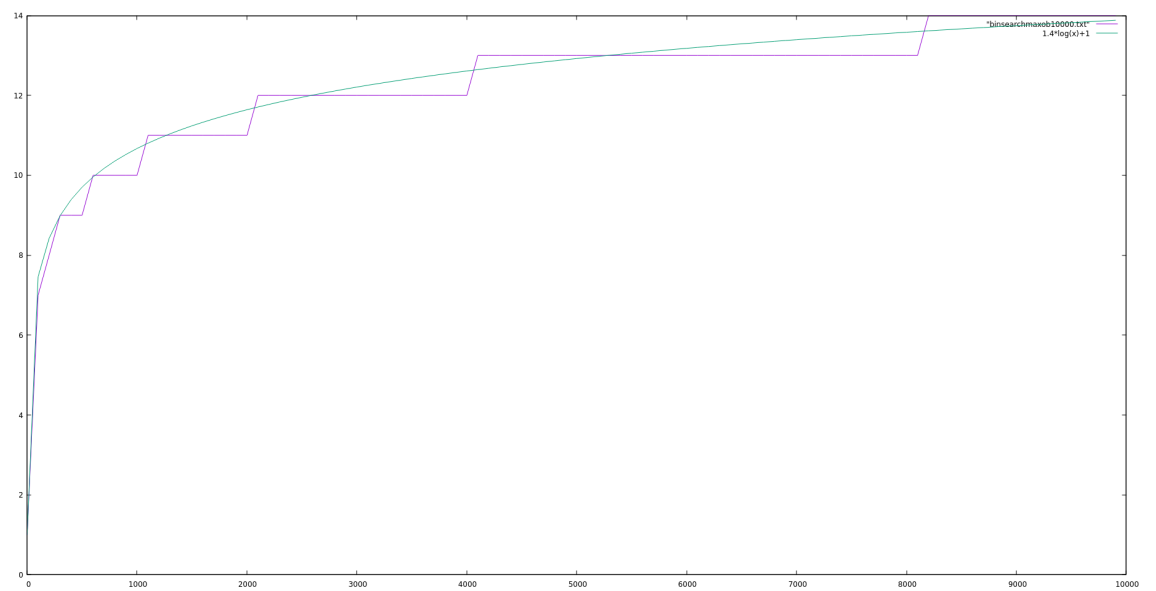
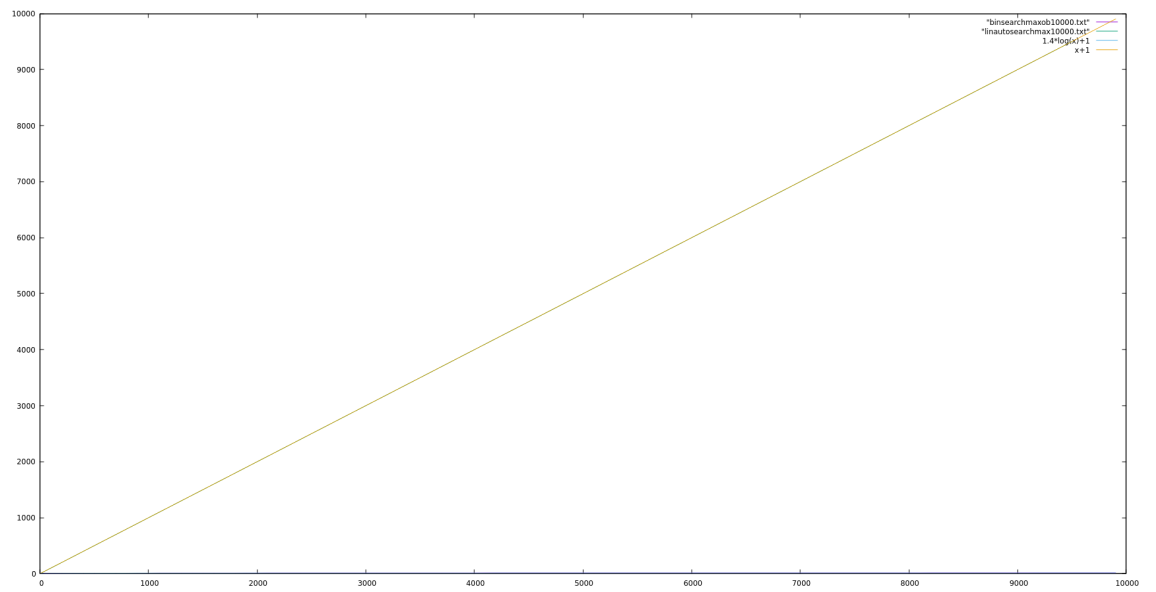
N_times 1:



N_times 100:



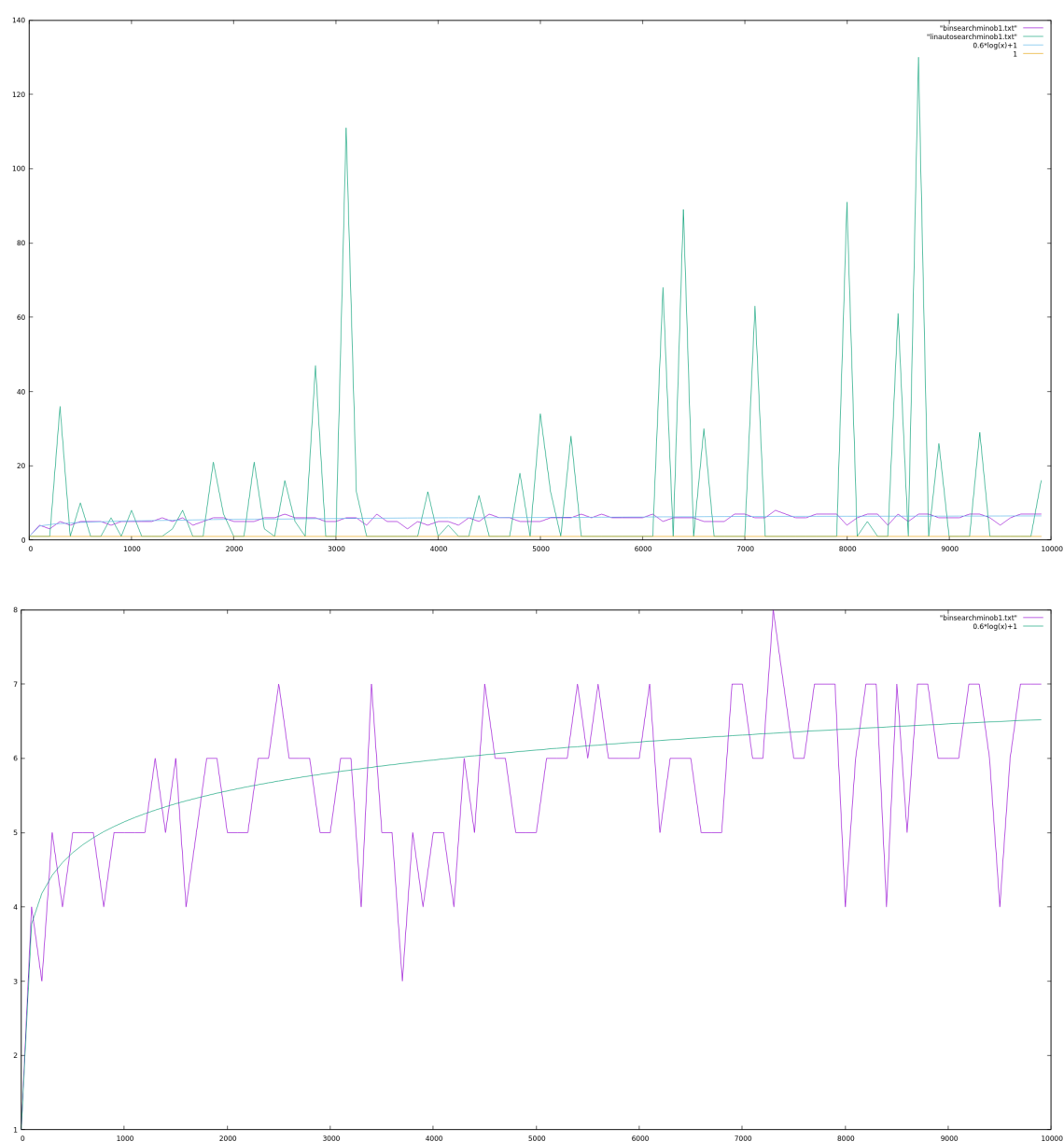
N_times 10000:

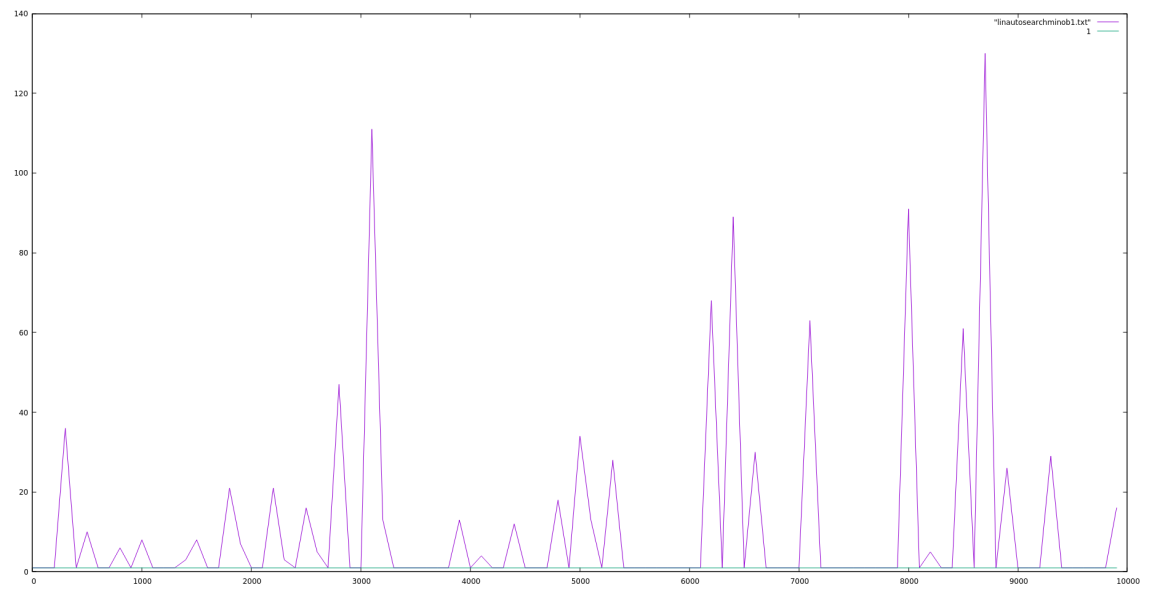


Como podemos observar, el número máximo de obs de la búsqueda binaria es el mismo para todos los distintos n_times , ya que la tabla no cambia. Para el lin_auto_search es similar, ya que esta no te asegura que para todos los números vaya a tardar menos sino que va a tardar menos para aquellos números que salen más frecuentemente. Es por eso que todas las gráficas del auto search son iguales o muy similares a la función N .

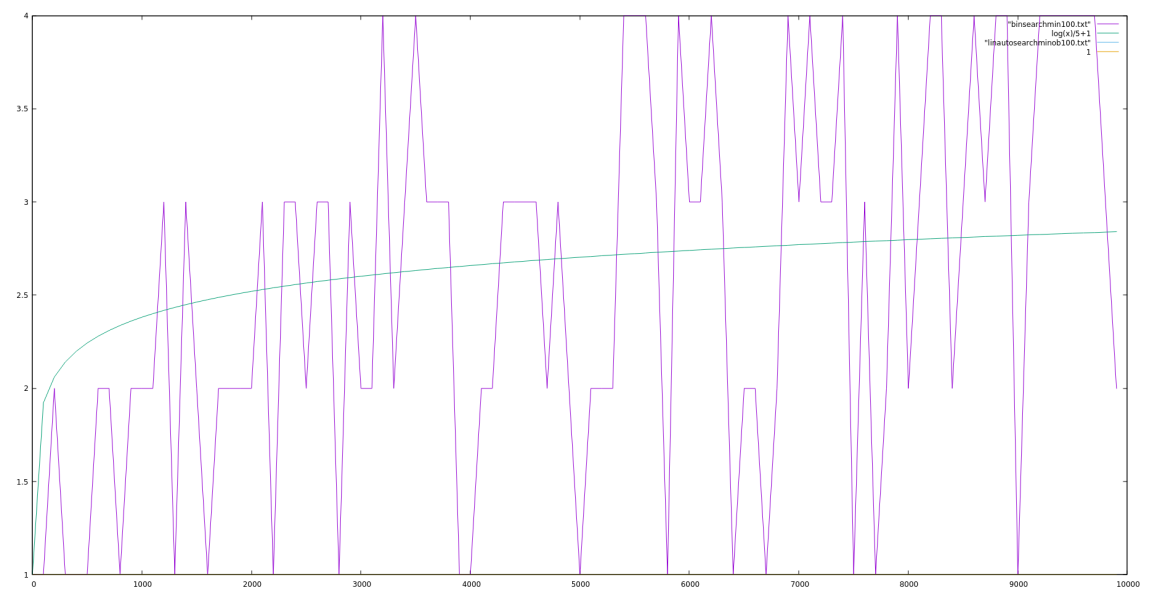
Gráfica comparando el número mínimo de OBs entre la búsqueda binaria y la búsqueda lineal auto organizada (para los valores de $n_times=1, 100$ y 10000), comentarios a la gráfica:

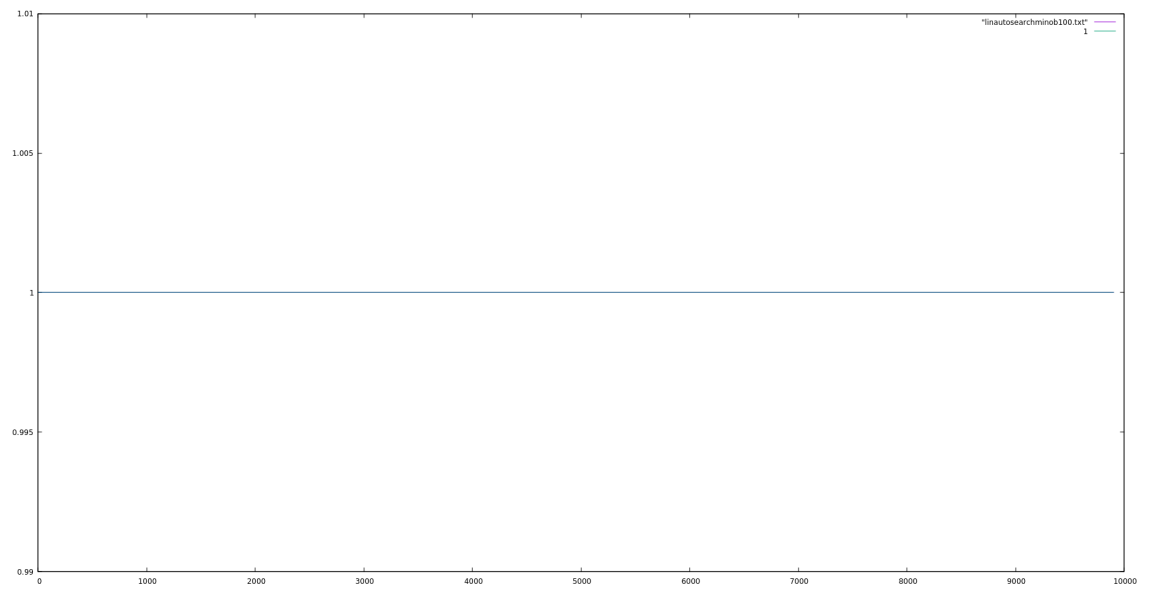
N_times 1:



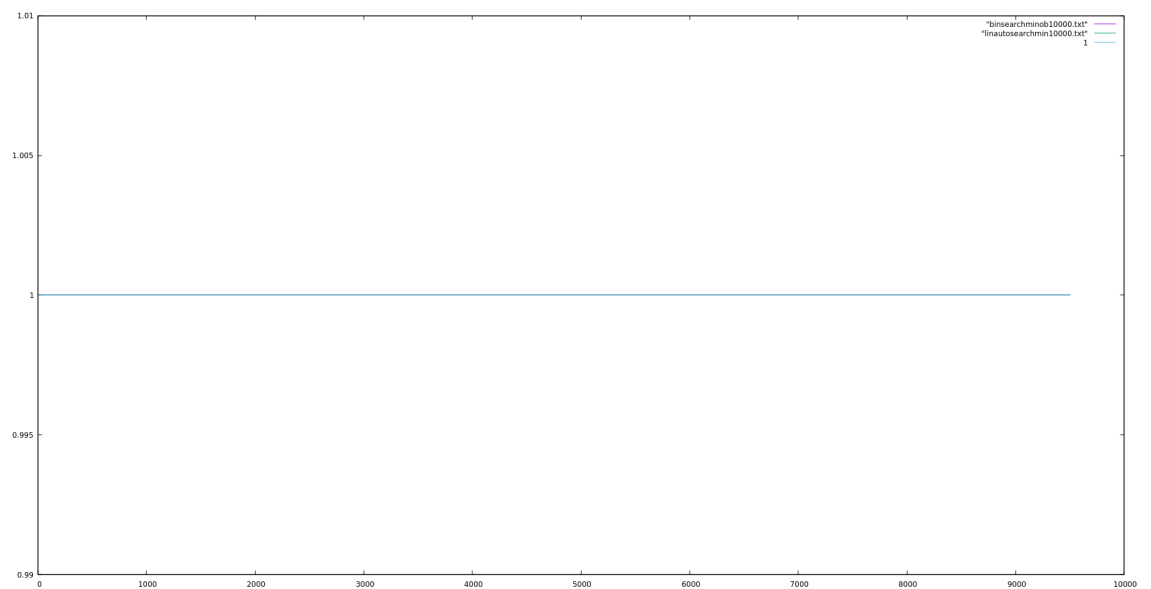


N_times 100:





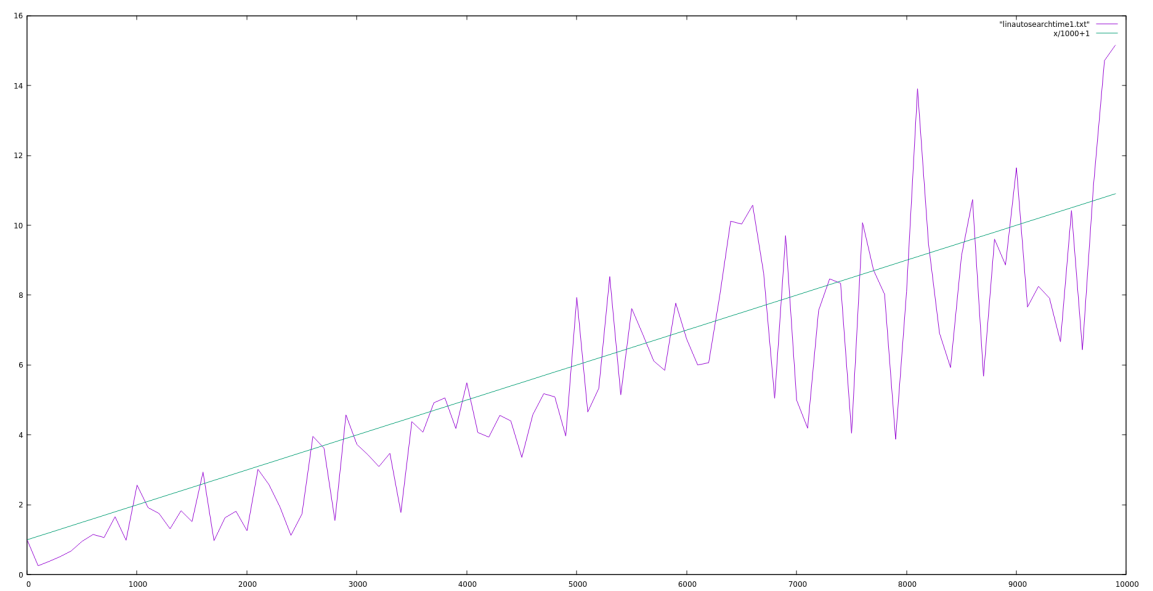
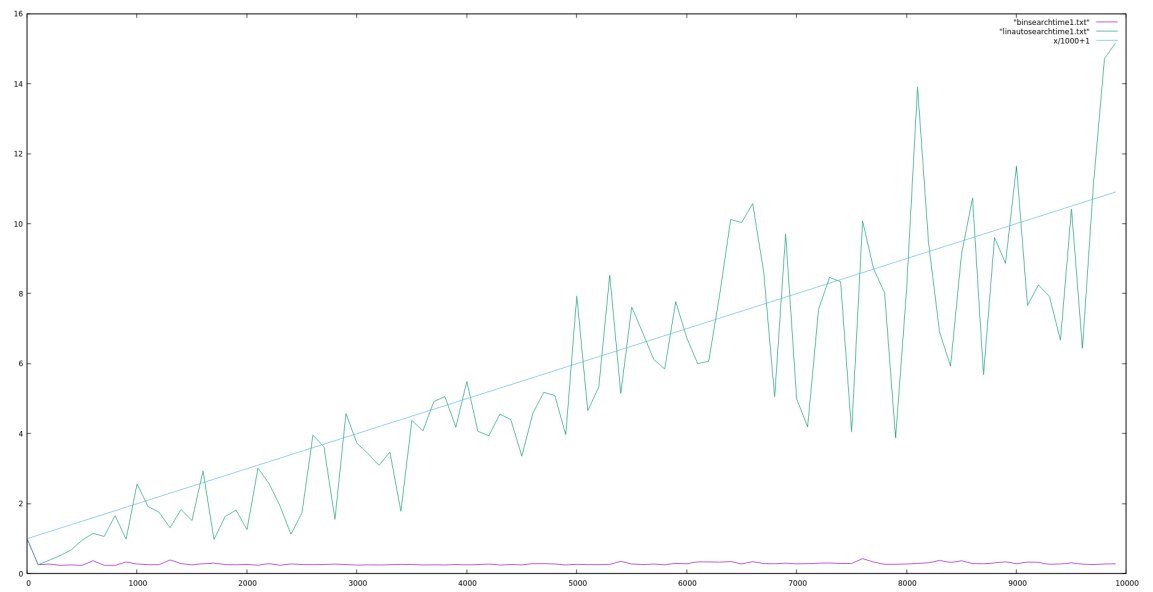
N_times 10000:



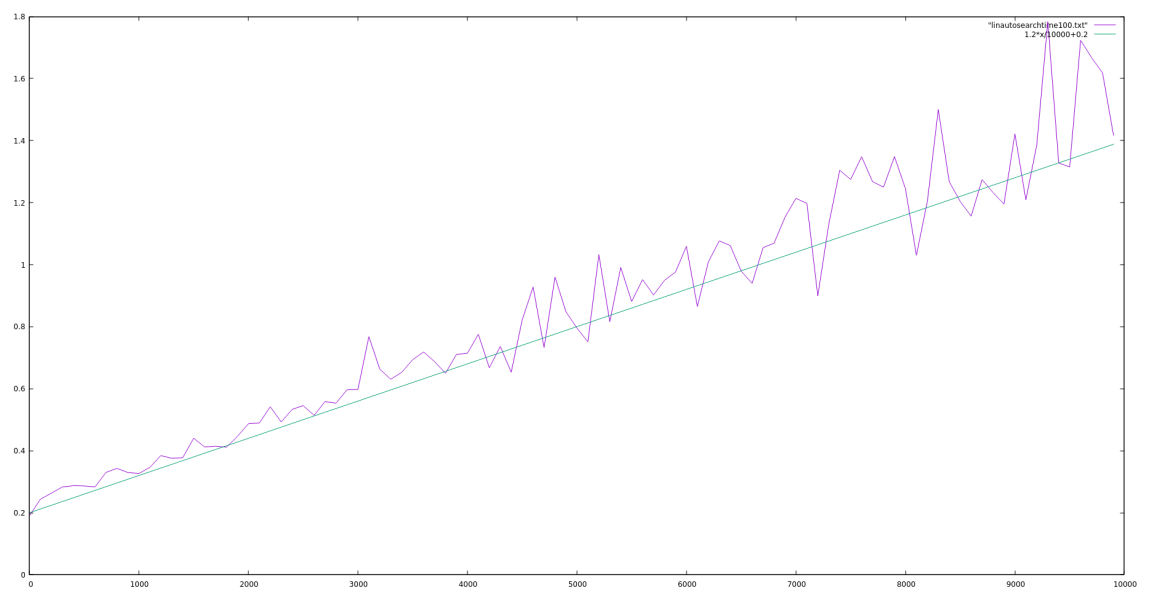
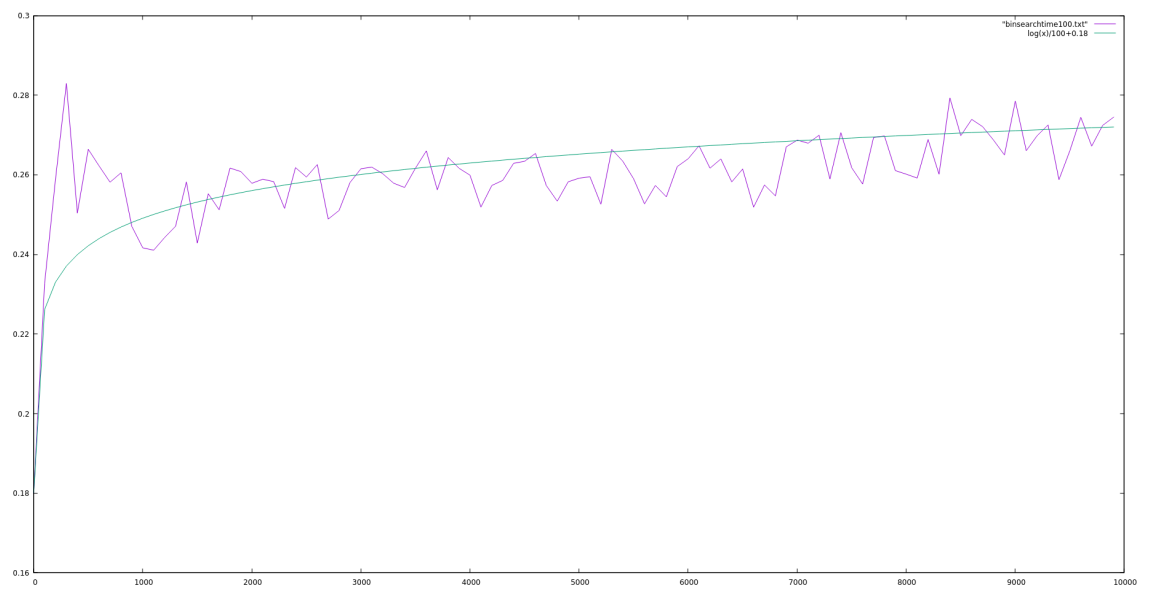
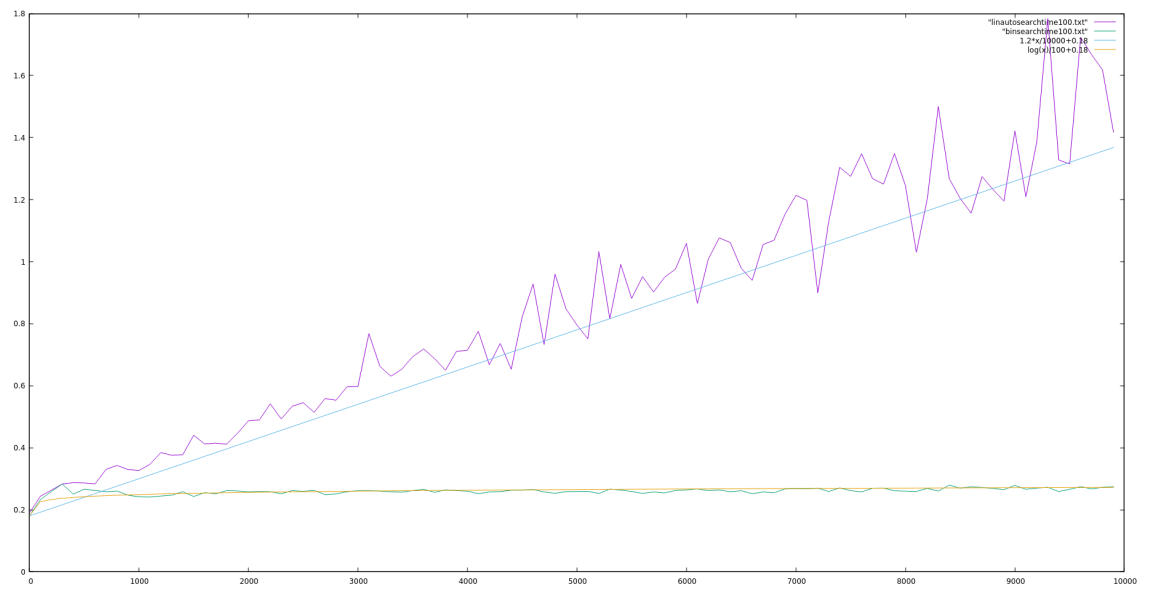
El número mínimo de obs de las dos funciones a la larga es el mismo (1) ya que es su caso mejor, sin embargo si ponemos un n_times pequeño nos salen picos ya que no siempre se cumple la cota más baja.

Gráfica comparando el tiempo medio de reloj entre la búsqueda binaria y la búsqueda lineal auto organizada (para los valores de n_times=1, 100 y 10000), comentarios a la gráfica:

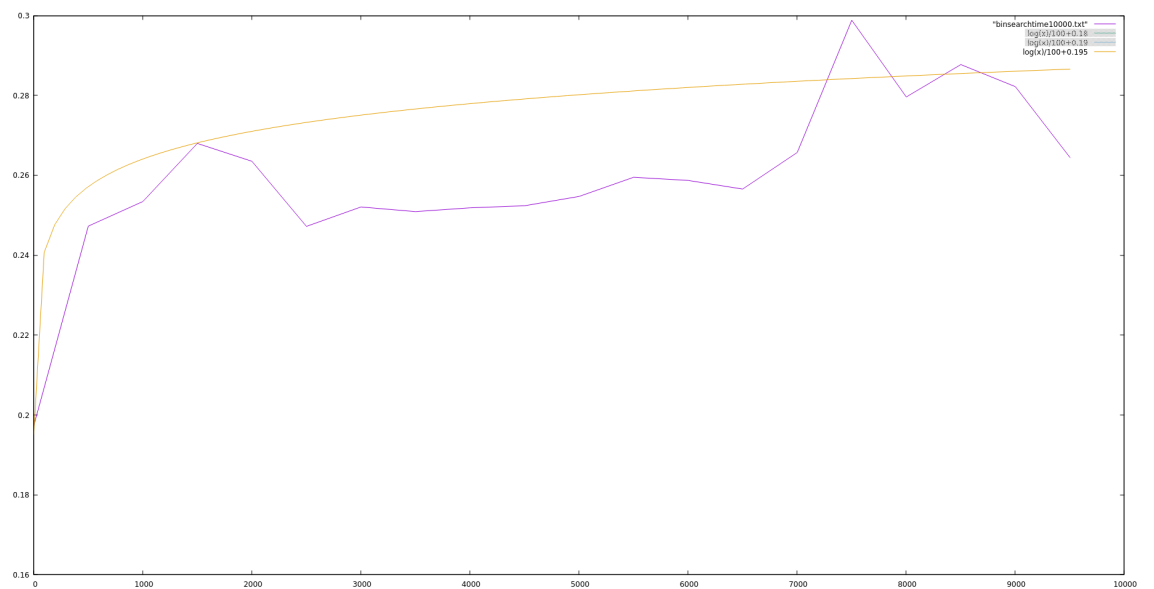
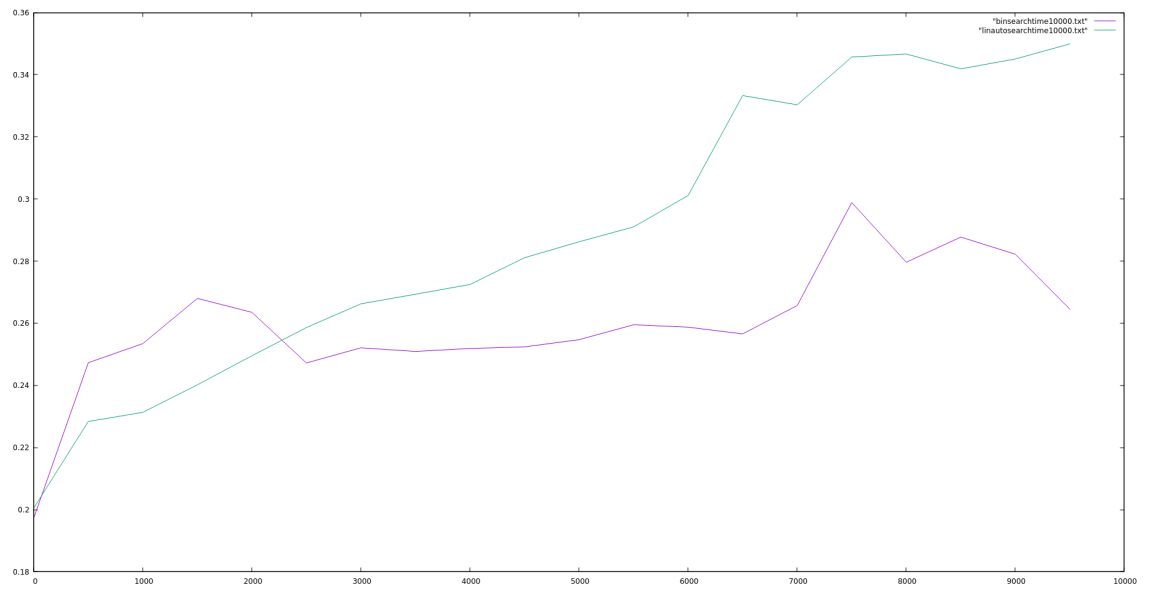
N_times 1:

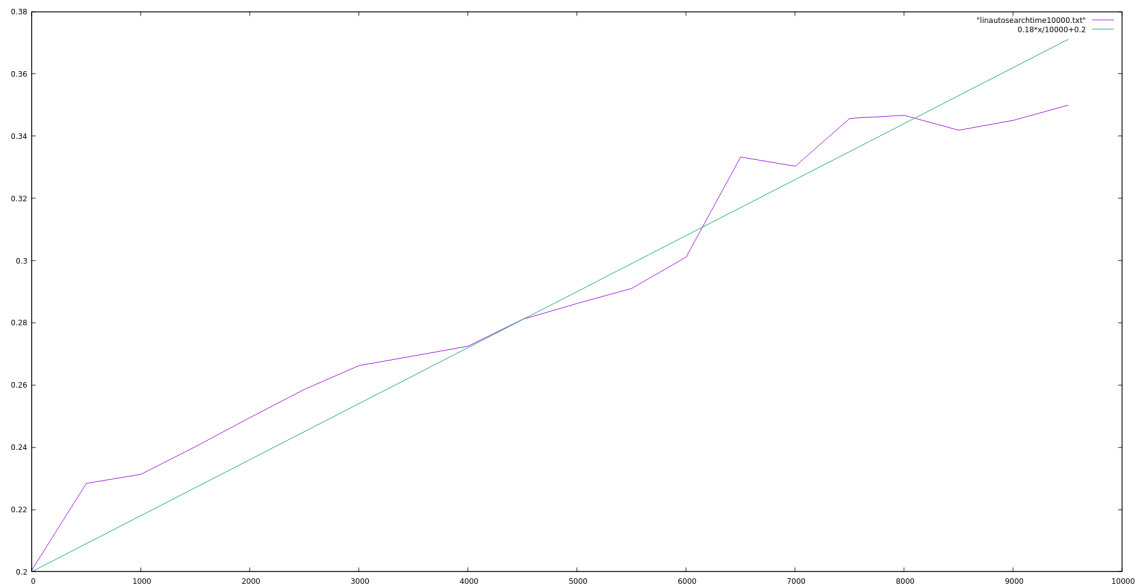


N_times 100:



N_times 10000:





Para el tiempo de estas dos funciones cuanto mayor sea el n_times mejor es el rendimiento de la búsqueda auto lineal. Esto se debe a que con la elección potencial de claves se van a ir eligiendo mayoritariamente los mismos números y al ir quedando estos números al principio de la tabla cada vez los encontraremos más rápido. La búsqueda binaria sin embargo no se ve afectada por el n_times porque la tabla siempre va a estar ordenada. En todos los casos que hemos probado es más eficiente la búsqueda binaria, sin embargo si pusieramos un n_times grandísimo la búsqueda auto lineal sería más eficiente.

5. Respuesta a las preguntas teóricas.

5.1 Pregunta 1

La operación básica de las tres búsquedas es la comparación de la clave y el elemento de la tabla, ya que se encuentra en el bucle más interno de éstas y es la que condiciona que el bucle pare o no.

5.2 Pregunta 2

El caso mejor de la búsqueda binaria y de la lineal son ambos $[O(1)]$ ya que ambos pueden dar con la clave a la primera. En el caso peor, es decir, que el elemento no esté en la tabla, la búsqueda lineal comparará antes esta clave con todos los elementos de la tabla, por lo que el coste será de $[N]$, sin embargo, la búsqueda binaria comparará con $[\lg(N) + O(1)]$ elementos ya que en caso de fallo, reducirá la tabla a la mitad.

5.3 Pregunta 3

Según se realizan más búsquedas, pueden pasar dos cosas, que si la elección de las claves es completamente aleatoria, no supondrá apenas cambios con respecto a la búsqueda lineal normal. Sin embargo, si en la elección de claves son más frecuentes unos números determinados, entonces estas claves serán encontradas de forma más rápida, y a medida que vayamos buscando más claves, menos tardará y los elementos que más se están buscando estarán cada vez más cercanos al inicio de la tabla.

5.4 Pregunta 4

Según el resultado de la pendiente de la gráfica que mide el número medio de obs para la búsqueda lineal autoorganizada, el resultado en función de N para un número razonable de claves buscadas($N \cdot n_times$) es aproximadamente de $N/160 + O(N)$.

5.5 Pregunta 5

El algoritmo `bin_search` busca bien las claves debido a que la tabla que hay que pasarle debe de estar ordenada, por tanto si comparas el número que está en el medio con la clave y te queda más pequeña pues está 100% seguro que el elemento estará en esa subtabla izquierda anterior al elemento del medio que hemos comparado, y lo mismo pasaría si fuera mayor, pero la subtabla derecha. Así dividiendo las tablas en subtablas llegaríamos a un resultado que siempre será correcto, porque estará entre los valores que hemos ido comparando.

6. Conclusiones finales.

En esta práctica hemos aprendido a hacer un TAD diccionario y sus correspondientes funciones para el correcto funcionamiento de este. También hemos creado las funciones de búsqueda de clave de búsqueda lineal, binaria y auto lineal. Tras varias tomas de datos, hemos llegado a la conclusión que la búsqueda lineal es sin duda la más costosa, y que la búsqueda auto lineal es buena sólo para elecciones de clave que no sean aleatorias, es decir que haya una serie de números que se repitan más que otros como clave, en caso contrario sería como una búsqueda lineal normal. Por último sabemos que la mejor es la búsqueda binaria aunque solo sirve para cadenas que estén ordenadas.